

计算机科学与技术学院

系统开发工具基础

第四周实验报告

学生姓名： 杜铭昊

学号： 25020007021

授课教师： 周小伟

实验环境： Ubuntu Linux 虚拟机

报告内容： 课堂第 13-16 题与 10 个扩展实例

实验日期： 2026 年 9 月 14 日

代码质量、Make 构建、API 处理与综合交付

目录

1 实验环境与证据规范	3
2 第 13 题：建立可执行的本地质量门禁	3
2.1 实验原理	3
2.2 步骤 1：复制并配置项目	3
2.3 步骤 2：补充行为测试	3
2.4 步骤 3：格式化、检查与测试	3
2.5 步骤 4：编写统一检查入口	4
2.6 结果与反思	4
3 第 14 题：让 Make 只重建真正受影响的产物	5
3.1 实验原理	5
3.2 步骤 1：建立源文件	5
3.3 步骤 2：编写准确 Makefile	5
3.4 步骤 3：验证首次构建和无操作构建	5
3.5 步骤 4：验证依赖传播与清理范围	5
3.6 结果与错误反思	6
4 第 15 题：把本地 API 数据转换为可读报告	7
4.1 实验原理	7
4.2 步骤 1：启动本地服务并获取数据	7
4.3 步骤 2：筛选与排序	7
4.4 步骤 3：生成 summary.md	7
4.5 结果与错误反思	8
5 第 16 题：修复并交付陌生的小型工具仓库	9
5.1 实验原理	9
5.2 步骤 1：制造并确认真实功能失败	9
5.3 步骤 2：定位并实施最小修复	9
5.4 步骤 3：建立 Make 交付入口	9
5.5 步骤 4：检查、构建与摘要	9
5.6 结果与错误反思	10
6 第四周扩展实例 1-10	11
6.1 实例 1：ruff 格式门禁	11
6.2 实例 2：静态检查自动修复	11
6.3 实例 3：参数化测试	12
6.4 实例 4：最小 Make 增量构建	12
6.5 实例 5：Make 干运行与依赖链	13
6.6 实例 6：并行 Make	13
6.7 实例 7：curl 错误退出契约	14
6.8 实例 8：jq 聚合统计	14
6.9 实例 9：CLI JSON 输出契约	15
6.10 实例 10：完整发布流水线	15
7 GitHub 分阶段提交记录	16

8 综合结论

17

1 实验环境与证据规范

实验在 Ubuntu 虚拟机中实测，Windows 用于文件同步和 GitHub 推送。主要工具包括 Python 3.12、ruff 0.16.5、pytest、GNU Make、curl、jq、build、Git 和 SHA-256 工具。虚拟机系统默认 Python 版本较旧，因此涉及 f-string 的步骤显式使用课程 Python 3.12。所有题目均保存源码、运行脚本、结果日志和 Linux 桌面截图；扩展实例每两个形成一次独立提交。

2 第 13 题：建立可执行的本地质量门禁

2.1 实验原理

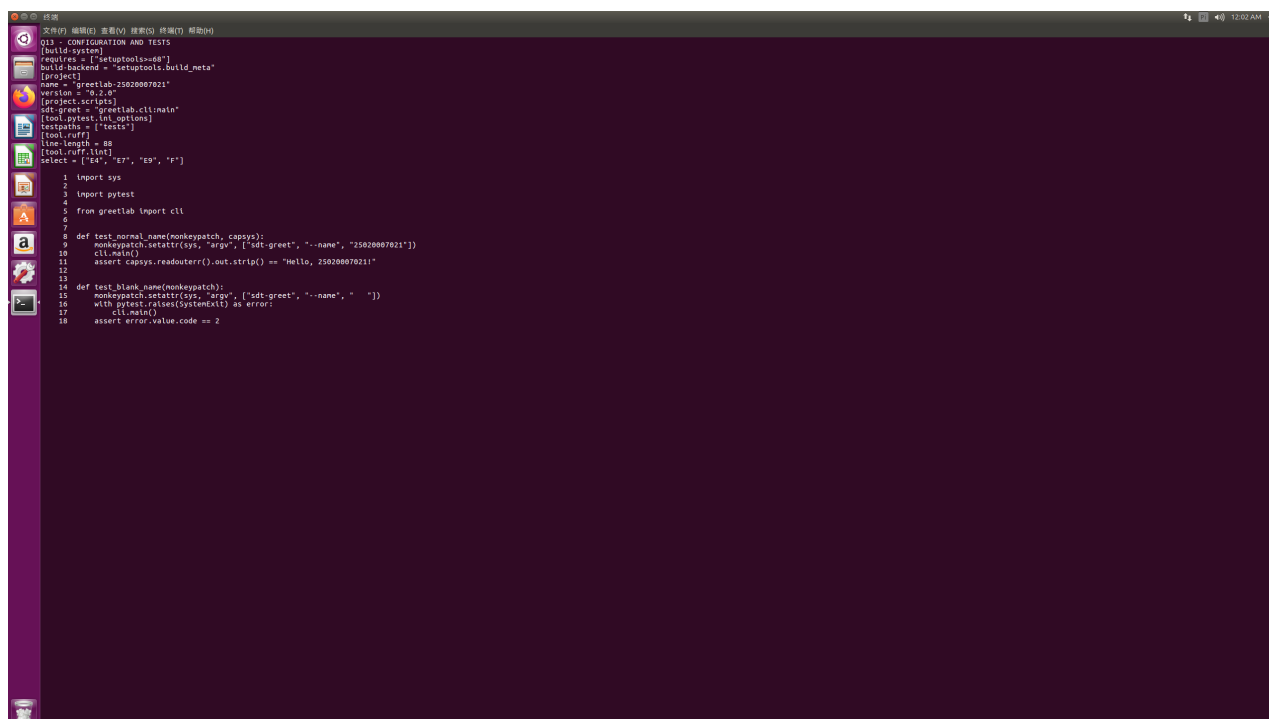
质量门禁把格式、静态检查和自动化测试组合为单一、可重复执行的入口。`ruff format --check` 保证格式稳定，`ruff check` 检测未使用导入和常见语法问题，`pytest` 验证外部行为。脚本采用 `set -euo pipefail`，任一环节失败都会产生非零退出码，从而阻止不合格代码进入后续构建。

2.2 步骤 1：复制并配置项目

以第 10 题 `greetlab` 为基础建立 `q13`，保留 `src/greetlab` 布局。在 `pyproject.toml` 中配置 `pytest` 测试目录、`ruff` 行宽和 E4、E7、E9、F 规则，没有使用全局忽略。

2.3 步骤 2：补充行为测试

正常姓名测试将参数设为学号，并断言标准输出严格等于 `Hello, 25020007021!`；空白姓名测试断言程序抛出 `SystemExit` 且退出码为 2。两个测试都包含有效断言。



```
q13 - CONFIGURATION AND TESTS
[build-system]
requires = ["setuptools<60"]
build-backend = "setuptools.build_meta"
[project]
name = "greetlab-25020007021"
version = "0.2.0"
[project.scripts]
sd-greet = "greetlab.cli:main"
[tool.pytest.ini_options]
testpaths = ["tests"]
[tool.ruff]
line-length = 88
[tool.ruff.lint]
select = ["E4", "E7", "E9", "F"]

1 import sys
2
3 import pytest
4
5 from greetlab import cli
6
7
8 def test_normal_name(monkeypatch, capsys):
9     monkeypatch.setattr(sys, 'argv', ['sd-greet', '--name', '25020007021'])
10    cli.main()
11    assert capsys.readouterr().out.strip() == "Hello, 25020007021!"
12
13
14 def test_blank_name(monkeypatch):
15     monkeypatch.setattr(sys, 'argv', ['sd-greet', '--name', ''])
16     with pytest.raises(SystemExit) as error:
17         cli.main()
18     assert error.value.code == 2
```

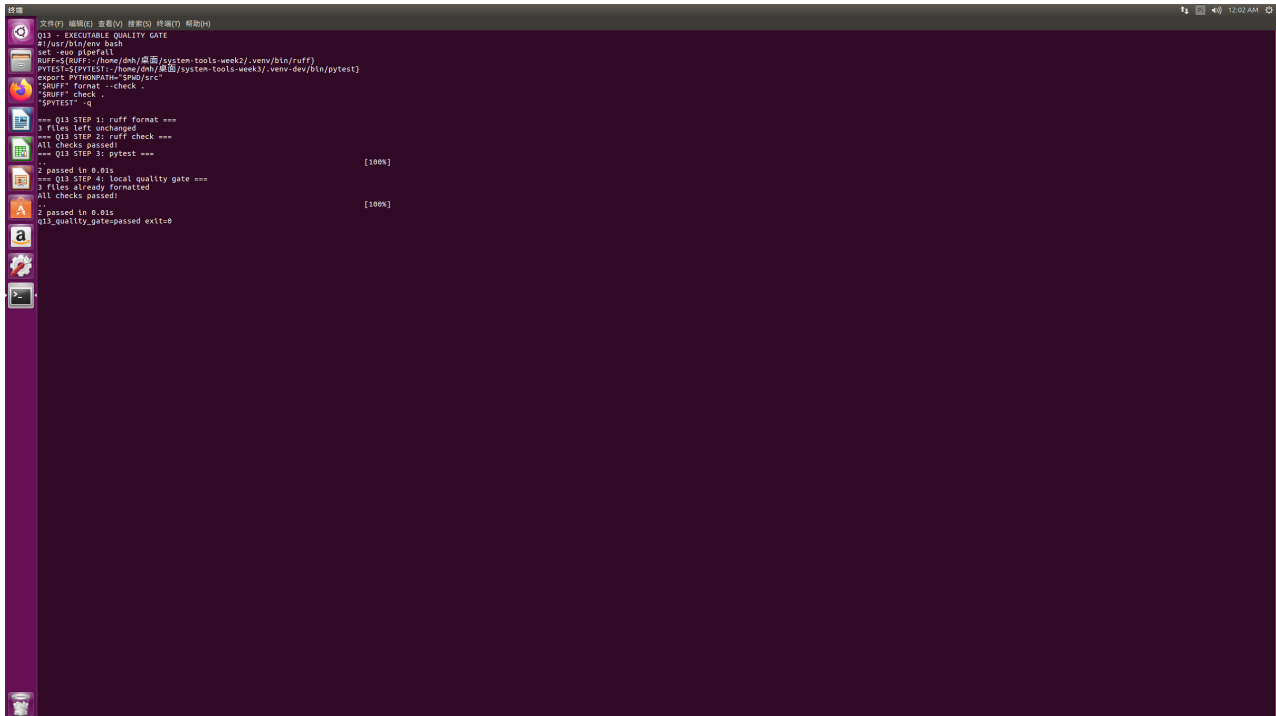
图 1: 第 13 题：项目配置与正常/空白姓名测试

2.4 步骤 3：格式化、检查与测试

依次运行 `ruff format`、`ruff check` 和 `pytest`。结果为 3 个文件格式正确、静态检查全部通过、2 个测试通过。

2.5 步骤 4: 编写统一检查入口

创建 `check.sh`, 依次执行格式只检查模式、静态检查和 `pytest`。实际运行返回 0, 并输出 `q13_quality_gate=p`



```
Q13 - EXECUTABLE QUALITY GATE
#!/usr/bin/env bash
set -eo pipefail
RUFF=$(RUFF:-/home/dmh/.venv/system-tools-week2/.venv/bin/ruff)
PYTEST=$(PYTEST:-/home/dmh/.venv/system-tools-week3/.venv-dev/bin/pytest)
export PYTHONPATH="$PWD/src"
"$RUFF" format --check .
"$RUFF" check .
"$PYTEST" -q

=== Q13 STEP 1: ruff format ===
3 files left unchanged
=== Q13 STEP 2: ruff check ===
All checks passed!
=== Q13 STEP 3: pytest ===
[100%]
2 passed in 0.01s
=== Q13 STEP 4: local quality gate ===
3 files already formatted
All checks passed
[100%]
2 passed in 0.01s
q13_quality_gate=passed exit:0
```

图 2: 第 13 题: `check.sh` 与质量门禁通过结果

2.6 结果与反思

本题所有门禁通过。单独执行工具容易遗漏步骤, 统一脚本使开发者和持续集成环境执行同一份质量契约; 不应通过全局忽略规则来“消除”错误, 而应修复根因。

3 第 14 题：让 Make 只重建真正受影响的产物

3.1 实验原理

Make 根据“目标、依赖和时间戳”决定是否执行配方。`stats.txt` 依赖数据和统计脚本；`report.txt` 依赖说明、统计中间产物和报告脚本。当上游数据变化时，依赖链应自动传递；无变化时应保持静默。`all` 与 `clean` 不是文件，必须声明为 `.PHONY`。

3.2 步骤 1：建立源文件

创建题目给定的 `data.csv`、`stats.py`、`build_report.py` 和 `report.md`。数据值 2、3、5 的合计应为 10。

3.3 步骤 2：编写准确 Makefile

默认目标 `all` 依赖 `report.txt`；统计目标依赖数据和统计脚本；报告目标依赖 Markdown、统计结果和报告脚本；`clean` 只删除两个生成物。由于系统默认 Python 无法解析题目中的 f-string，Makefile 明确指定 `/home/dmh/.local/bin/python3.12`。

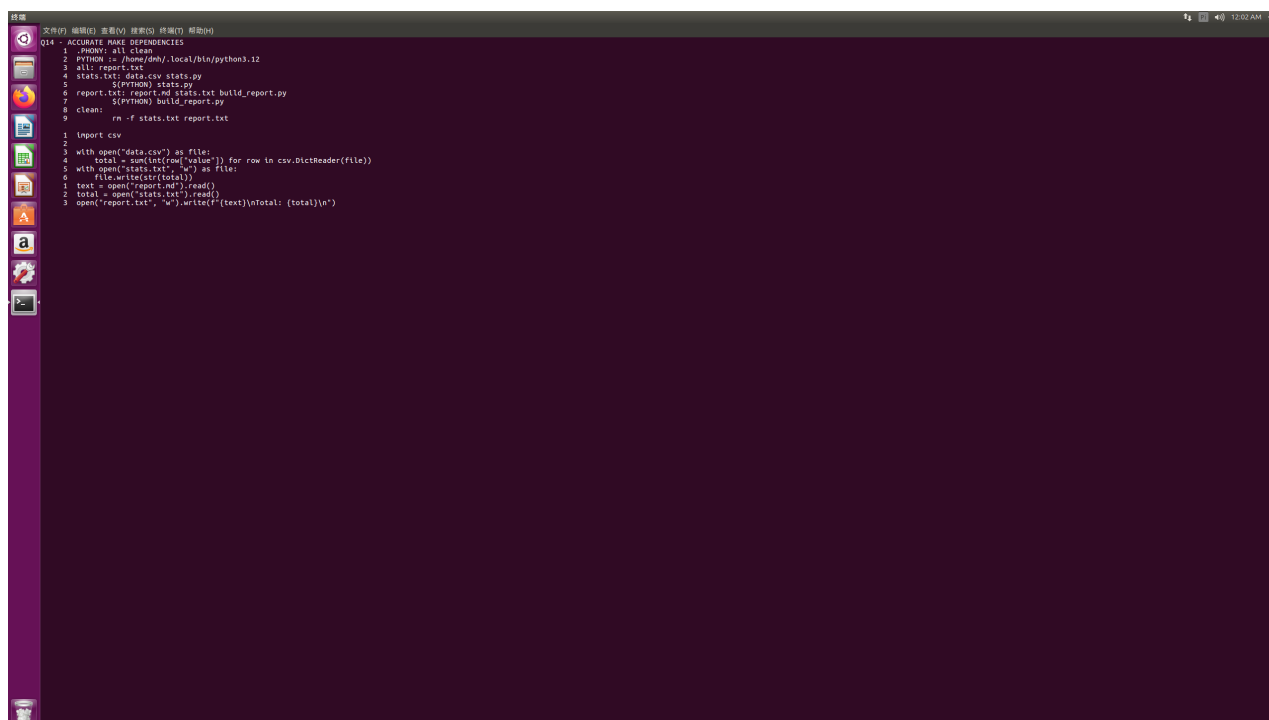


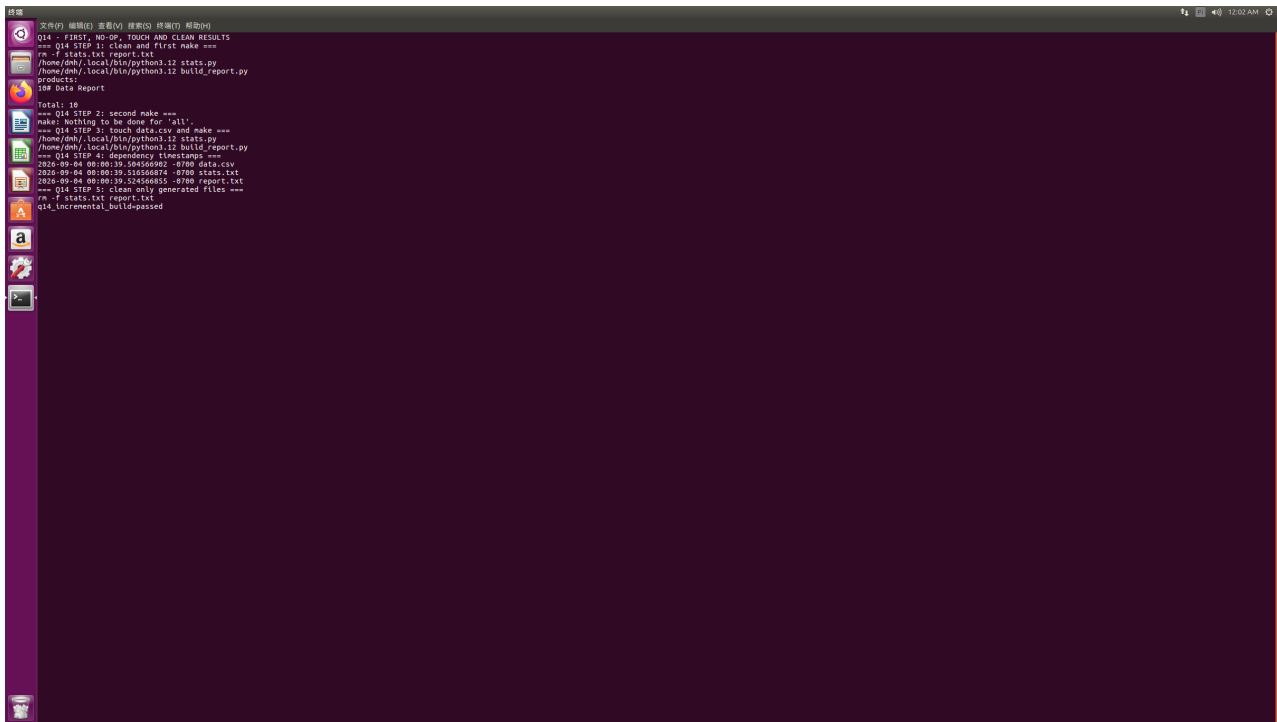
图 3: 第 14 题：Makefile 依赖关系与处理脚本

3.4 步骤 3：验证首次构建和无操作构建

第一次 `make` 依次执行统计和报告配方，得到统计值 10 与报告文本；第二次无修改时输出 `Nothing to be done for 'all'`，说明未做无效重建。

3.5 步骤 4：验证依赖传播与清理范围

执行 `touch data.csv` 后，`stats` 和 `report` 按顺序重新生成，时间戳严格递增。执行 `clean` 后，数据、脚本和 Markdown 仍存在，只有生成物被删除。



```
Q14 - FIRST, NO-OP, TOUCH AND CLEAN RESULTS
=== Q14 STEP 1: clean and first make ===
rm -f stats.txt report.txt
/home/dmh/.local/bin/python3.12 stats.py
/home/dmh/.local/bin/python3.12 build_report.py
products:
158 Data Report

Total: 18
=== Q14 STEP 2: second make ===
make: Nothing to be done for 'all'.
=== Q14 STEP 3: touch data.csv and make ===
/home/dmh/.local/bin/python3.12 stats.py
/home/dmh/.local/bin/python3.12 build_report.py
=== Q14 STEP 4: dependency timestamps ===
2820-09-04 08:00:39.546568602 ->780 data.csv
2820-09-04 08:00:39.516566874 ->780 stats.txt
2820-09-04 08:00:39.546568602 ->780 report.txt
=== Q14 STEP 5: clean only generated files ===
rm -f stats.txt report.txt
q14_incremental_build-passed
```

图 4: 第 14 题：首次构建、无操作、touch 重建和 clean 结果

3.6 结果与错误反思

增量构建验证通过。首次运行出现 `SyntaxError`，根因是系统默认 Python 过旧而非 Make 依赖错误；修复方法是解释器作为显式构建变量，避免“同名命令、不同运行时”导致不可复现。

4 第 15 题：把本地 API 数据转换为可读报告

4.1 实验原理

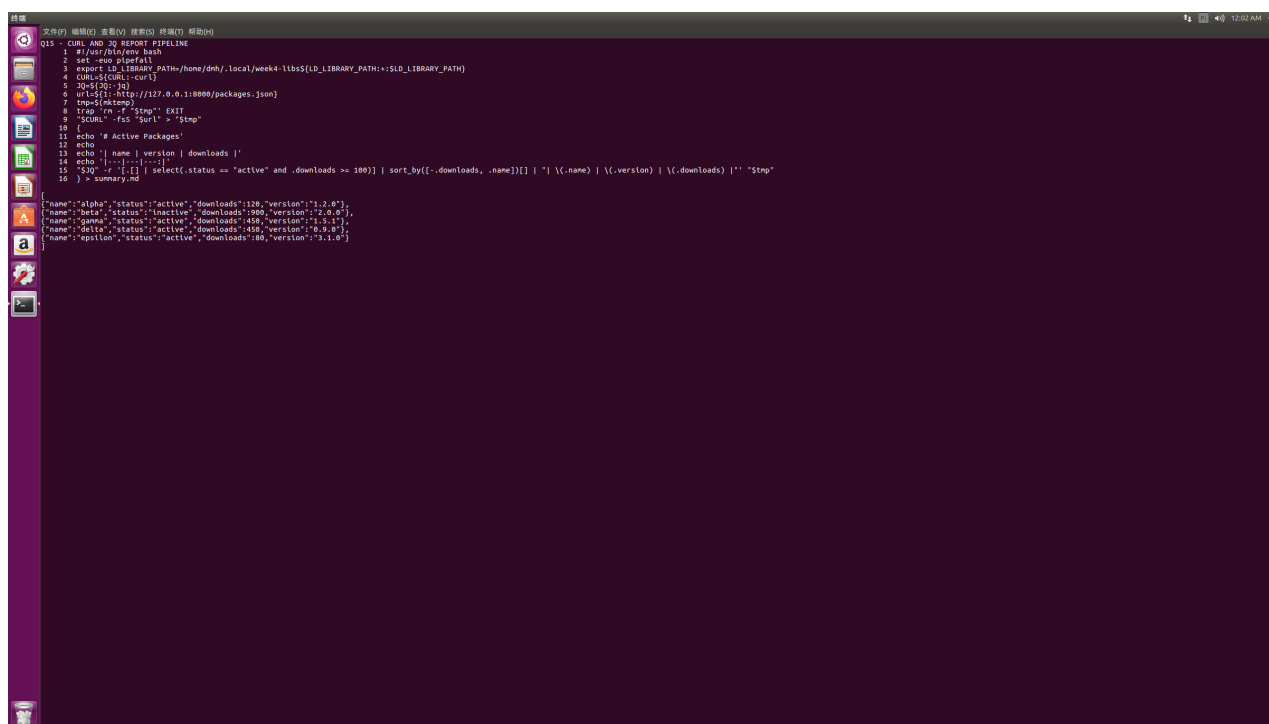
本题形成“HTTP 获取–JSON 过滤–排序–Markdown 呈现”的流水线。curl 的 -f 使 HTTP 错误返回非零，-sS 减少正常噪声但保留错误；jq 执行结构化过滤，避免用文本切割破坏 JSON 语义。排序键先使用 downloads 降序，再使用 name 升序解决并列。

4.2 步骤 1：启动本地服务并获取数据

保存 5 条软件包 JSON，在 q15 目录运行 `python3 -m http.server 8000`。用 curl 访问本机地址，jq 读取数组长度为 5，证明服务和 JSON 均有效。

4.3 步骤 2：筛选与排序

筛选 status 为 active 且 downloads 不少于 100 的记录，得到 delta、gamma、alpha。delta 与 gamma 下载量同为 450，按 name 升序时 delta 在前，符合双键排序要求。



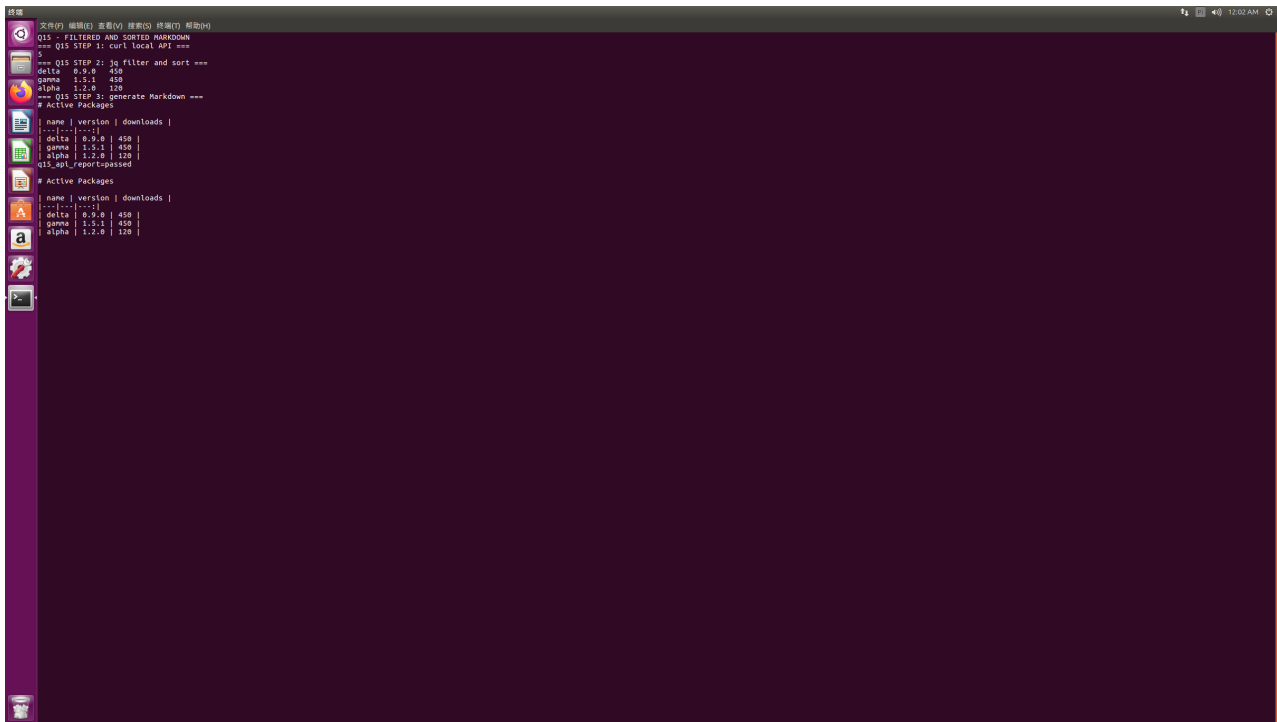
```
Q15 ~ curl AND jq REPORT PIPELINE
1 #!/usr/bin/env bash
2 set -eu pipefail
3 export LD_LIBRARY_PATH=/home/dnh/.local/week4-libs:${LD_LIBRARY_PATH}
4 CURS=$(curl -sS)
5 JQ=$(jq -c)
6 WQS=$(curl -sS http://127.0.0.1:8000/packages.json)
7 tmp=$(mktemp)
8 trap 'rm -f "$tmp"' EXIT
9 "curl" -sS "curl" > "$tmp"
10 [
11   echo "# Active Packages"
12   echo
13   echo -n "name | version | downloads |"
14   echo "-----"
15   "$WQS" -r '[?].status == "active" and .downloads >= 100] | sort_by(-.downloads, .name)|[]' | "jq -r '{name: .name, version: .version, downloads: .downloads}'" > "$tmp"
16 ] > summary.md

{"name":"alpha","status":"active","downloads":120,"version":"1.2.0"},
{"name":"beta","status":"inactive","downloads":990,"version":"2.0.0"},
{"name":"gamma","status":"active","downloads":450,"version":"1.5.1"},
{"name":"delta","status":"active","downloads":450,"version":"0.9.0"},
{"name":"epsilon","status":"active","downloads":80,"version":"3.1.0"}
```

图 5: 第 15 题：curl、jq 与 Markdown 生成脚本

4.4 步骤 3：生成 summary.md

`api_report.sh` 使用临时文件保存响应，并通过 trap 清理；输出标题及 name、version、downloads 三列表格。最终表中依次为 delta 0.9.0 450、gamma 1.5.1 450、alpha 1.2.0 120。



```
Q15 - FILTERED AND SORTED MARKDOWN
=== Q15 STEP 1: curl local API ===
5

=== Q15 STEP 2: jq filter and sort ===
delta 0.9.0 450
gamma 1.1.1 450
alpha 1.2.0 120

=== Q15 STEP 3: generate Markdown ===

# Active Packages

| name | version | downloads |
|---|---|---|
| delta | 0.9.0 | 450 |
| gamma | 1.1.1 | 450 |
| alpha | 1.2.0 | 120 |
| q15_q1_reportpassed
```

图 6: 第 15 题: 过滤排序结果与 summary.md

4.5 结果与错误反思

API 报告生成通过。虚拟机最初缺少 curl 和 jq, 且无无密码 sudo 权限; 因此从 Ubuntu 官方软件源下载相应软件包并仅解压到用户目录, 未改变系统级配置。脚本显式设置工具路径和 jq 动态库路径, 保证他人可以理解运行环境。

5 第 16 题：修复并交付陌生的小型工具仓库

5.1 实验原理

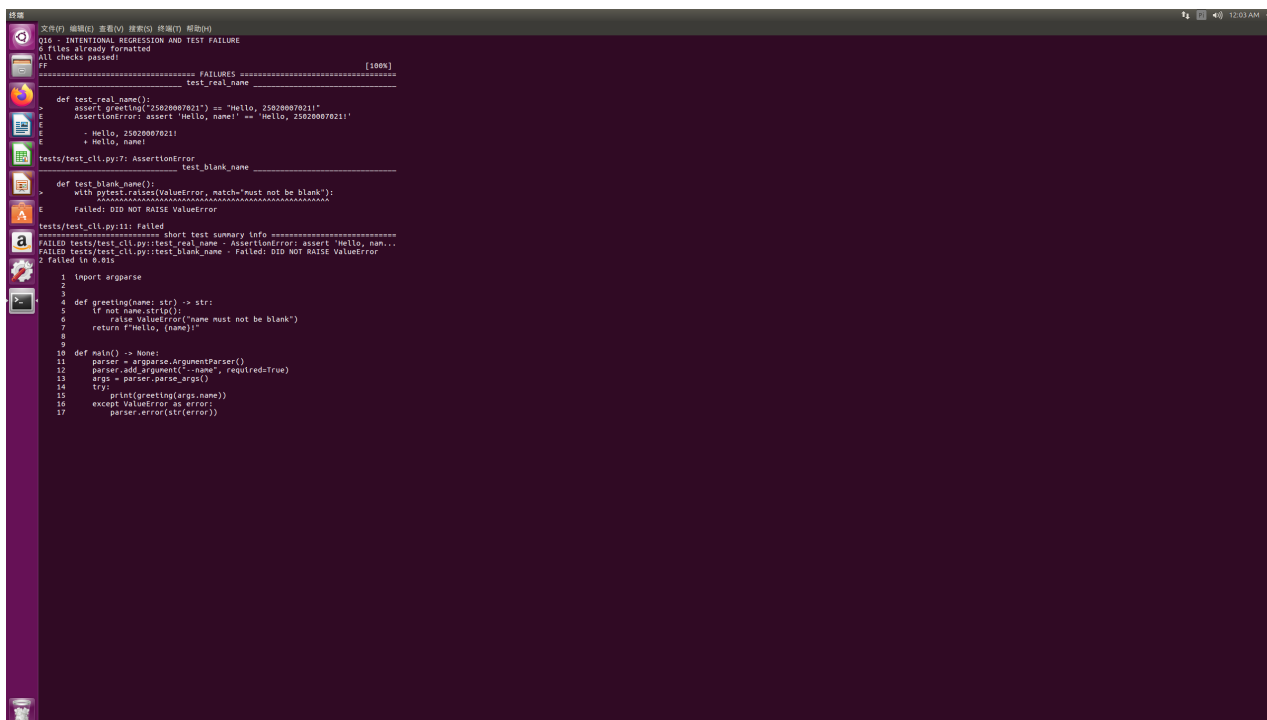
综合交付要求先用测试捕获回归，再完成最小修复，并通过统一入口产生可校验制品。Makefile 把 check 作为 build 的前置条件，从结构上保证“不通过检查就不构建”；wheel 的 SHA-256 摘要用于识别交付制品。

5.2 步骤 1：制造并确认真实功能失败

复制 q13 为 q16，把问候函数临时改为始终返回字面量 `Hello, name!`。先格式化故障代码，避免格式门禁提前终止；随后运行 check，ruff 通过但两个 pytest 失败：真实姓名输出错误，空白姓名没有抛出异常，退出码为 1。

5.3 步骤 2：定位并实施最小修复

恢复基于参数构造问候语的实现，并在空白输入时抛出 `ValueError`，CLI 将其转换为参数错误。没有引入无关重构。



```
Q16 - INTENTIONAL REGRESSION AND TEST FAILURE
4 files already formatted
All checks passed!
#
===== FAILURES ===== [100%]
test_real_name
def test_real_name():
    assert greeting('25020007021') == 'Hello, 25020007021!'
E
AssertionError: assert 'Hello, name!' == 'Hello, 25020007021!'
E
- Hello, 25020007021!
E
+ Hello, name!
tests/test_cli.py:7: AssertionError
test_blank_name
def test_blank_name():
    with pytest.raises(ValueError, match='must not be blank'):
E
Failed: DID NOT RAISE ValueError
tests/test_cli.py:11: Failed
===== short test summary info =====
FAILED tests/test_cli.py::test_real_name - AssertionError: assert 'Hello, nam...
FAILED tests/test_cli.py::test_blank_name - Failed: DID NOT RAISE ValueError
2 failed in 0.01s

1 import argparse
2
3
4 def greeting(name: str) -> str:
5     if not name.strip():
6         raise ValueError("name must not be blank")
7     return f'hello, {name}!'
8
9
10 def main() -> None:
11     parser = argparse.ArgumentParser()
12     parser.add_argument('--name', required=True)
13     args = parser.parse_args()
14     try:
15         print(greeting(args.name))
16     except ValueError as error:
17         parser.error(str(error))
```

图 7：第 16 题：故意回归、测试失败与最终实现

5.4 步骤 3：建立 Make 交付入口

Makefile 包含 check、build、clean 三个伪目标。check 调用 `check.sh`；build 依赖 check，清理旧产物后以 Python 3.12 调用 build 生成 wheel；clean 删除构建缓存和制品。

5.5 步骤 4：检查、构建与摘要

make check 得到 ruff 通过和 2 个测试通过；make build 成功生成 `greetlab_25020007021-0.3.0-py3-none` 随后计算并保存 SHA-256。

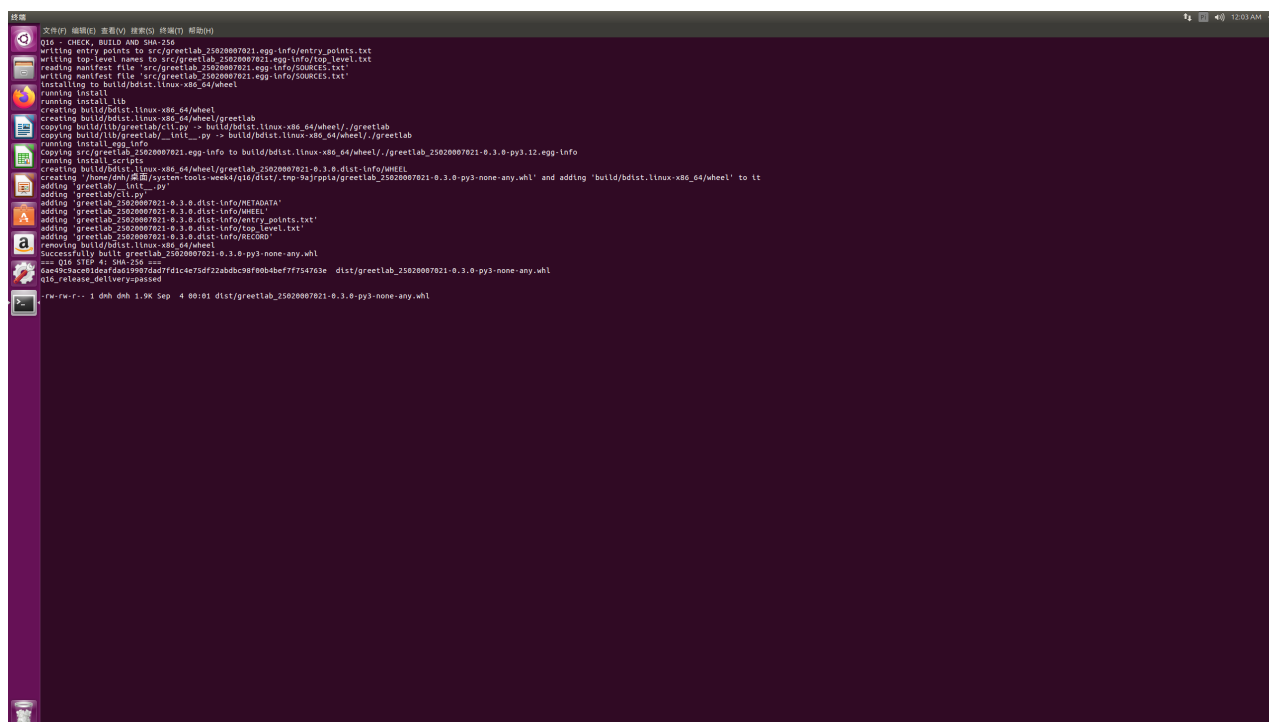


图 8: 第 16 题: 质量检查、wheel 构建与 SHA-256 结果

5.6 结果与错误反思

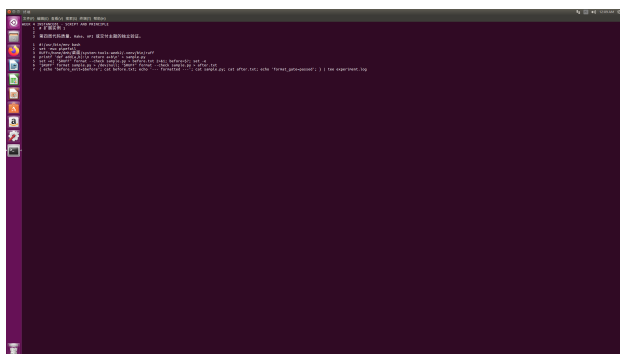
综合交付链通过。最初的故障演示被格式检查提前拦截，无法证明测试确实捕获功能缺陷；调整为先格式化故障代码后再执行 check，最终观察到两个真实功能测试失败。说明验证实验时必须确保失败来自目标缺陷，而不是更早的无关门禁。

6 第四周扩展实例 1-10

扩展实例围绕代码质量、自动测试、Make 依赖、并行构建、HTTP 错误契约、jq 统计和发布流水线设计。每个实例均在 Ubuntu 执行，并保存脚本与结果截图。

6.1 实例 1: ruff 格式门禁

故意创建缩进和逗号格式不规范的函数，`ruff format --check` 先返回 1 并显示差异；执行格式化后再次检查通过。原理是把代码风格转换为可执行约束。



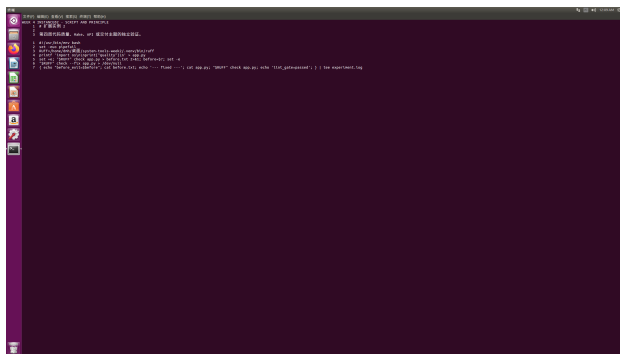
实例 1: 脚本与步骤



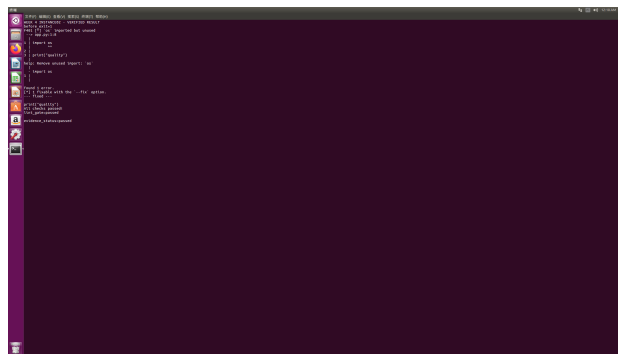
实例 1: 失败、修复和通过

6.2 实例 2: 静态检查自动修复

在程序中加入未使用的 `import os`, ruff 报告 F401 并返回 1；使用安全修复删除导入，最终 All checks passed。



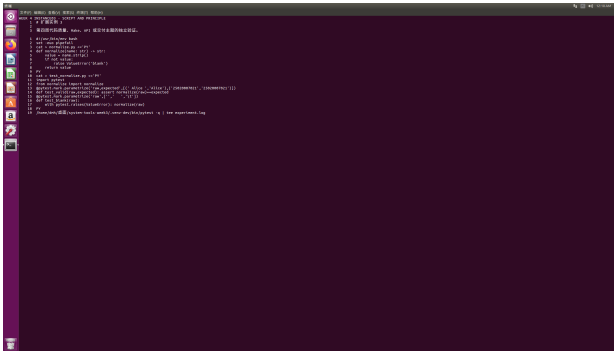
实例 2: 静态检查脚本



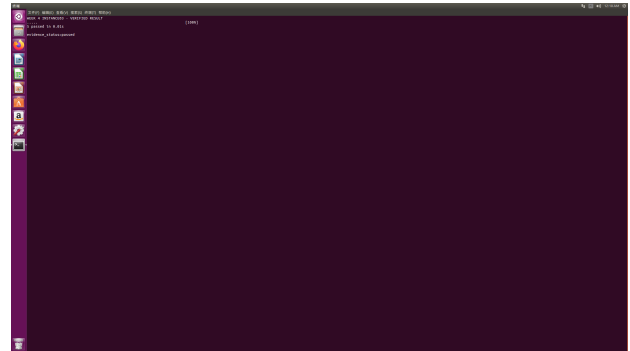
实例 2: F401 与修复结果

6.3 实例 3：参数化测试

实现姓名去空格与空值拒绝，通过 `pytest` 参数化覆盖 2 个正常输入和 3 个异常输入，最终 5 个测试全部通过。参数化可在不复制测试结构的情况下扩展边界覆盖。



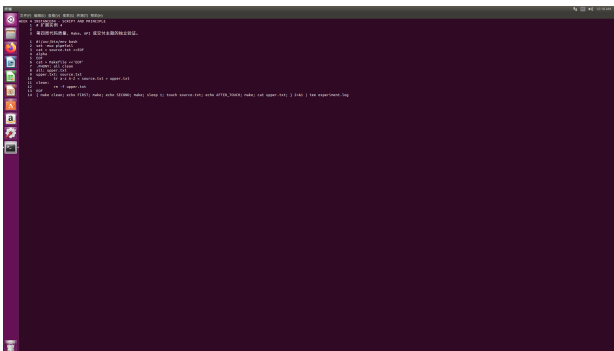
实例 3：参数化测试过程



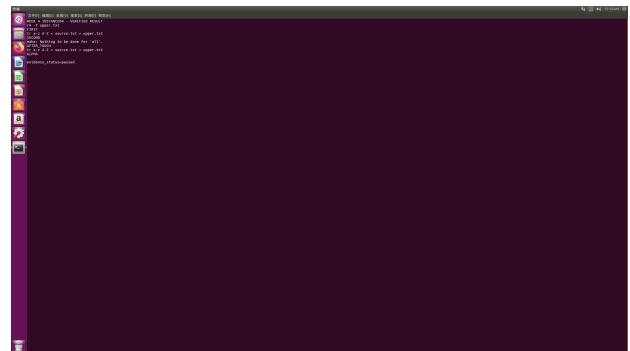
实例 3：5 项测试通过

6.4 实例 4：最小 Make 增量构建

首次生成大写文件，第二次 `make` 不执行配方；`touch` 源文件后目标重新生成并得到 ALPHA，证明时间戳依赖生效。



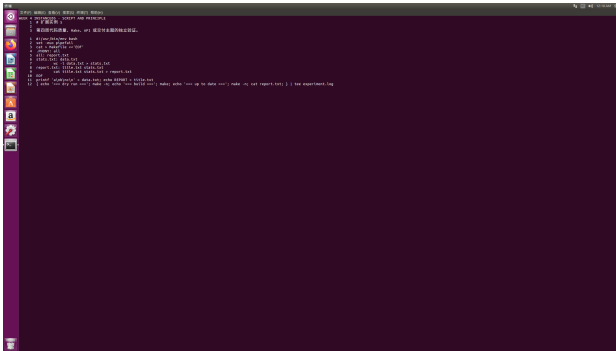
实例 4：Makefile 与步骤



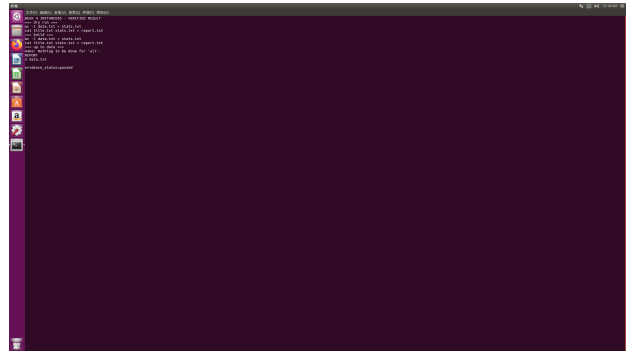
实例 4：增量构建结果

6.5 实例 5: Make 干运行与依赖链

使用 `make -n` 预览两条配方，正式构建后再次干运行显示无任务；报告正确包含标题和 3 行统计结果。干运行适合在不改动文件时审计构建计划。



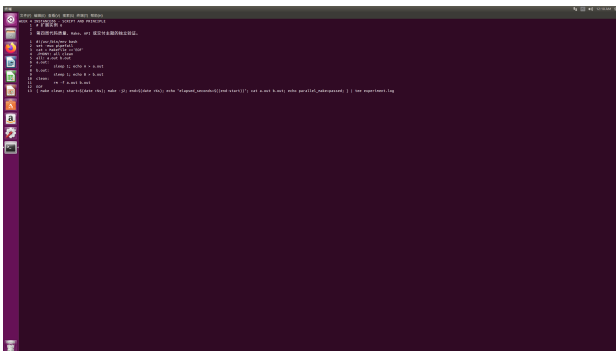
实例 5: 依赖链脚本



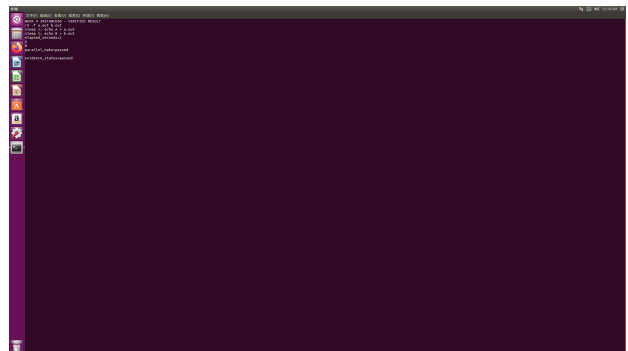
实例 5: 干运行和产物

6.6 实例 6: 并行 Make

两个互不依赖目标各等待 1 秒，`make -j2` 并行执行，总耗时约 1 秒且生成 A、B 两个文件，说明依赖图也决定可并行程度。



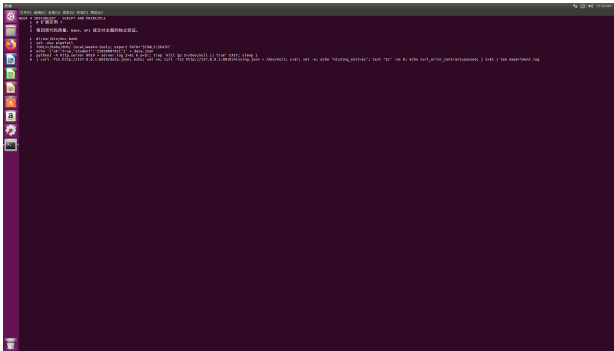
实例 6: 并行目标



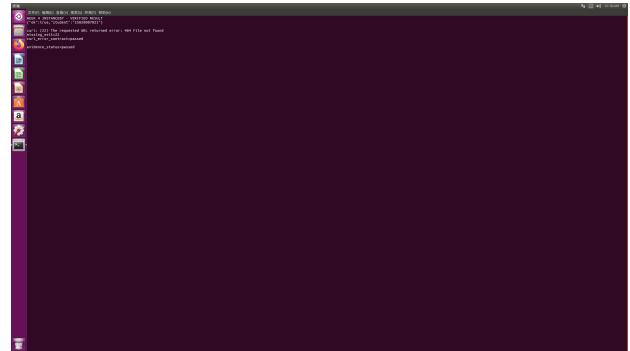
实例 6: 并行结果

6.7 实例 7: curl 错误退出契约

本地服务正常资源返回 JSON；不存在资源因 -f 得到 HTTP 404 和退出码 22，脚本断言其非零。网络脚本不仅要验证成功路径，也要让错误可被上层自动识别。



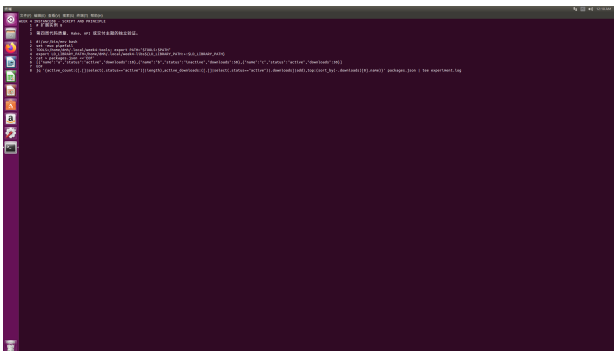
实例 7: HTTP 测试脚本



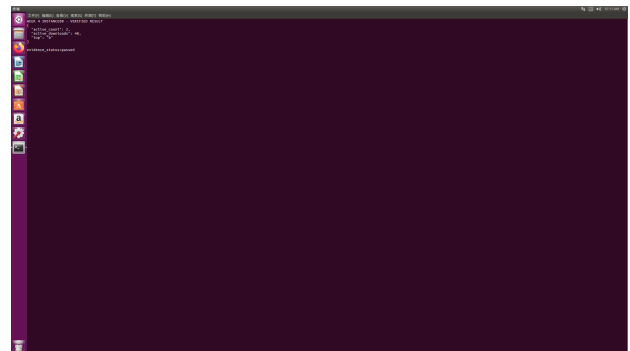
实例 7: 正常响应与 404 退出码

6.8 实例 8: jq 聚合统计

对 JSON 一次性计算活动记录数 2、活动下载总数 40 和总体下载最高项 b。结构化聚合避免多次解析或手工统计。



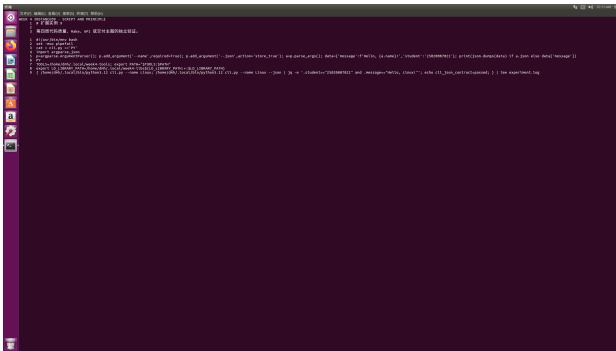
实例 8: jq 聚合命令



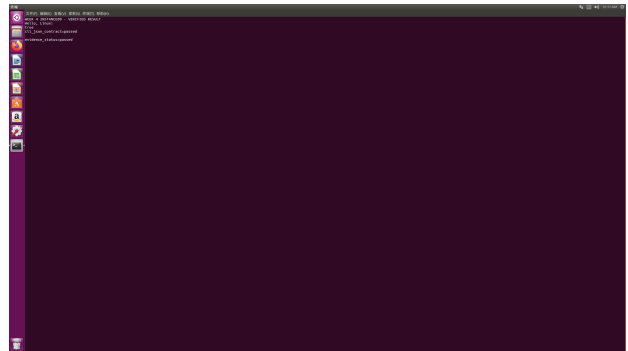
实例 8: JSON 统计结果

6.9 实例 9: CLI JSON 输出契约

CLI 同时支持文本与 JSON 输出;使用 jq 断言学号和消息字段,返回 true 并显示 `cli_json_contract=passed` 首次使用系统默认 Python 出现 f-string 语法错误,改为 Python 3.12 后通过。



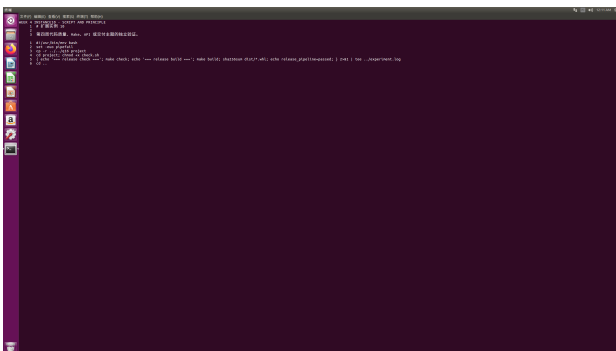
实例 9: CLI 脚本



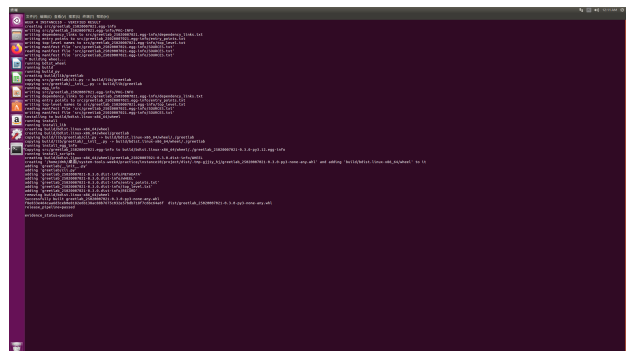
实例 9: jq 契约通过

6.10 实例 10: 完整发布流水线

复用 q16 项目,依次执行质量检查、wheel 构建和摘要计算;2 个测试、ruff 及构建全部通过,最终输出 `release_pipeline=passed`。



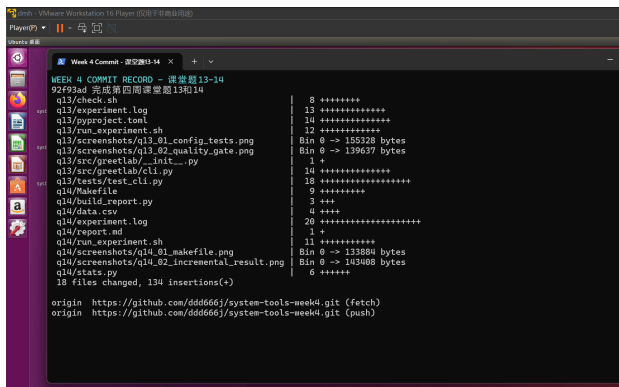
实例 10: 发布流水线



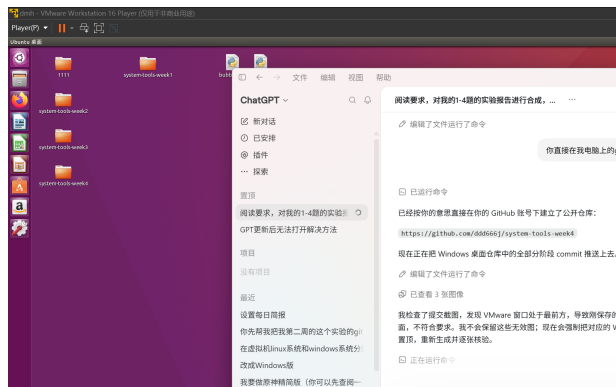
实例 10: 检查、构建与摘要

7 GitHub 分阶段提交记录

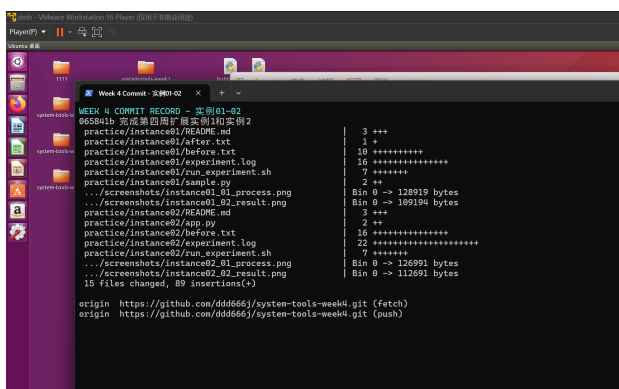
仓库地址为<https://github.com/ddd666j/system-tools-week4>。课堂题 13-14、15-16 分别提交；扩展实例每两个一次提交，最后单独提交记录文件。主要提交为 92f93ad、18cef9f、065841b、8c994fd、ef775de、073fe9c、0de0300 和 1c33c63。



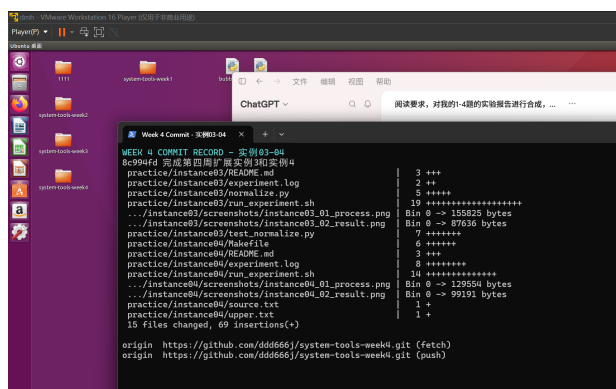
课堂题 13-14 提交



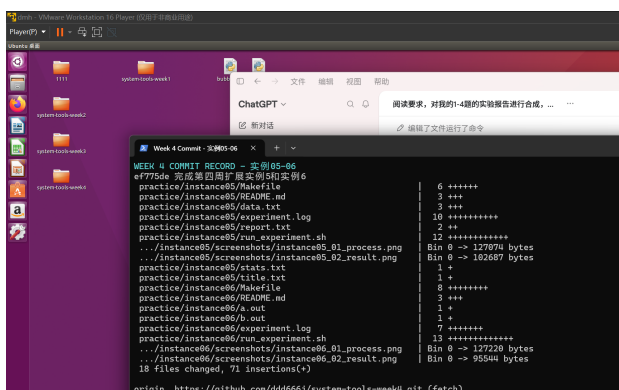
课堂题 15-16 提交



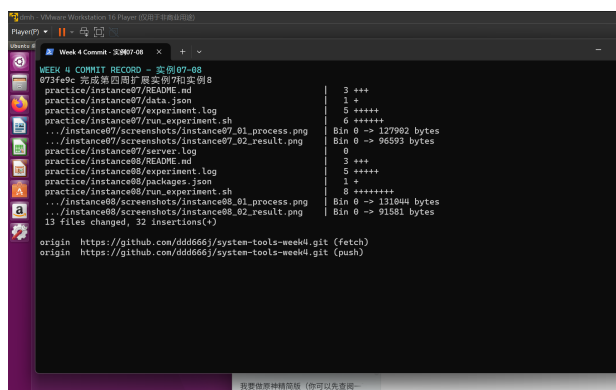
实例 1-2 提交



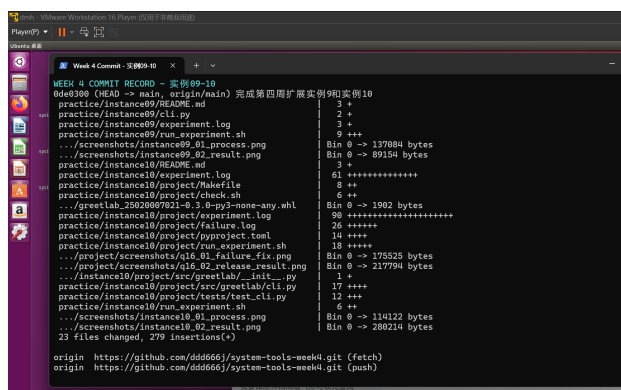
实例 3-4 提交



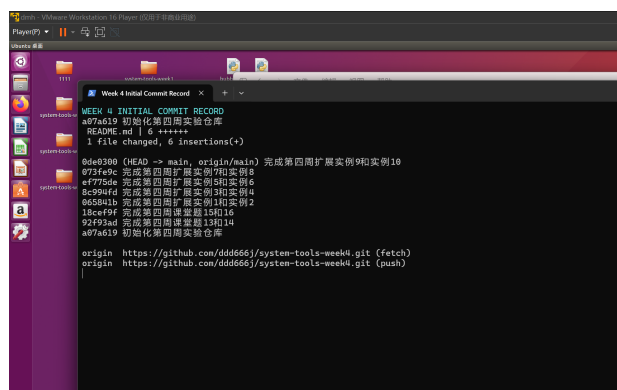
实例 5-6 提交



实例 7-8 提交



实例 9-10 提交



仓库初始化与完整历史

8 综合结论

本周完成了从代码质量、自动测试到增量构建、API 数据处理和 wheel 交付的完整工具链。实验中遇到的主要问题是默认 Python 版本不兼容、虚拟机缺少 curl 和 jq、功能失败被格式门禁提前遮蔽。解决原则是保留失败证据、定位实际失败层级、显式固定运行时与工具路径，并用最小修改重新验证。最终所有课堂题和 10 个扩展实例均在 Linux 实测通过，源码、日志、截图和 Git 历史已同步到 Windows 与公开 GitHub 仓库。